

EPROG Lernblatt – Serie 10

Klassen · Magic Methods · try/except · Testen

Was lernst du hier? Drei Dinge:

- **Klassen** – eigene Datentypen bauen (wie ein eigener Lego-Bauplan)
- **Magic Methods** – damit `+`, `*`, `print()` mit deinen Klassen funktionieren
- **try/except** – Fehler elegant abfangen statt Absturz

TEIL 1: Klassen und Objekte

ANALOGIE: Stell dir eine Klasse wie ein **Keksausstecher** vor. Der Ausstecher (= Klasse) definiert die **Form**. Jeder Keks, den du damit ausstichst (= Objekt), hat dieselbe Form, aber kann unterschiedlich dekoriert sein (= verschiedene Attribut-Werte).

Was ist was?

Begriff	Was ist das?	Beispiel
Klasse	Der Bauplan	<code>class Polynom:</code>
Objekt	Ein konkretes Ding	<code>p = Polynom([1,2,3])</code>
Attribut	Eigenschaft (= Variable im Objekt)	<code>self.grad = 2</code>
Methode	Fähigkeit (= Funktion im Objekt)	<code>def eval(self, x):</code>
Konstruktor	Was beim Erstellen passiert	<code>def __init__(self):</code>
self	Zeigt auf das aktuelle Objekt	<code>self.koeff = [1,2,3]</code>

Der Konstruktor: `__init__`

Wird **automatisch** aufgerufen wenn du ein Objekt erstellst. Hier packst du alles rein, was das Objekt **von Anfang an wissen muss**.

```
class Polynom:
    def __init__(self, koeffizienten):
        self.koeff = koeffizienten # speichern
        self.grad = len(koeffizienten) - 1 # automatisch berechnen
```

```
p = Polynom([1, 0, 3]) # → ruft __init__ auf
print(p.koeff) # [1, 0, 3]
print(p.grad) # 2
```

KEY TAKEAWAY: `self` = "ich selbst". Jede Methode bekommt als erstes `self`. Beim **Aufrufen** schreibst du es aber **nicht** hin: `p.eval(2)`, nicht `p.eval(p, 2)`.

Methoden = Funktionen im Objekt

Genau wie normale Funktionen, aber mit `self` als erstem Parameter:

```
def eval(self, x):
    ergebnis = 0
    for i, a in enumerate(self.koeff):
        ergebnis += a * x**i
    return ergebnis
```

Aufruf: `p.eval(2)` → Python macht daraus intern `Polynom.eval(p, 2)`.

Statische Attribute & Methoden

Gehören zur **Klasse**, nicht zum einzelnen Objekt:

```
class Student:
    anzahl = 0 # statisch: gehört zur Klasse

    def __init__(self, name):
        self.name = name # instanz: gehört zum Objekt
        Student.anzahl += 1 # über KLASSENNAME zugreifen!

    @staticmethod # Dekorator nicht vergessen!
    def get_anzahl():
        return Student.anzahl # kein self!
```

ACHTUNG: Statisch → Zugriff immer über `Klassenname.attribut`. Änderungen betreffen **alle** Objekte gleichzeitig.

Zugriffskontrolle: `_`, `__`

Schreibweise	Bedeutung	Kann man trotzdem zugreifen?
<code>self.name</code>	Public – für alle	Ja, ganz normal
<code>self._name</code>	Protected – bitte nicht anfassen	Ja, aber man sollte nicht
<code>self.__name</code>	Private – versteckt	Nur über <code>_Klasse__name</code>

MERKE: `__init__` ist KEIN privates Attribut! Doppelte Unterstriche **vorne UND hinten** = Magic Method. Doppelte Unterstriche **nur vorne** = privat.

TEIL 2: Magic Methods

ANALOGIE: Stell dir vor, Python fragt dein Objekt: *"Hey, jemand hat + gedrückt. Was soll ich tun?"*
Deine `__add__`-Methode ist die **Antwort** darauf. Ohne sie weiß Python nicht, was `p1 + p2` bedeuten soll.

Die wichtigsten Magic Methods auf einen Blick:

Du schreibst...	Python ruft auf...	Wozu?
<code>print(obj)</code>	<code>__str__(self)</code>	Schöne Textausgabe
<code>repr(obj)</code> / in Liste	<code>__repr__(self)</code>	Technische Darstellung
<code>obj1 + obj2</code>	<code>__add__(self, other)</code>	Addition definieren
<code>3 + obj</code>	<code>__radd__(self, other)</code>	Addition von links
<code>obj * 3</code>	<code>__mul__(self, other)</code>	Multiplikation (rechts)
<code>3 * obj</code>	<code>__rmul__(self, other)</code>	Multiplikation (links)
<code>obj1 < obj2</code>	<code>__lt__(self, other)</code>	Vergleich: kleiner
<code>obj1 <= obj2</code>	<code>__le__(self, other)</code>	Vergleich: kleiner-gleich
<code>obj1 >= obj2</code>	<code>__ge__(self, other)</code>	Vergleich: größer-gleich
<code>sorted(liste)</code>	braucht <code>__lt__</code>	Sortierung ermöglichen

`__str__` vs. `__repr__` – wann was?

	<code>__str__</code>	<code>__repr__</code>
Wann?	<code>print(obj)</code> , <code>str(obj)</code>	<code>repr(obj)</code> , in Listen
Zweck	Für Menschen lesbar	Für Debugging / Listen
Beispiel	<code>"[0, 4]"</code>	<code>"[0, 4]"</code>

ACHTUNG: Wenn du `print(sorted(liste))` machst, ruft Python `__repr__` auf, **NICHT** `__str__`!
Für Aufgabe 5 brauchst du **beide**.

Beispiel: `__add__` für Polynome

Problem: Polynome können **verschiedene Grade** haben. Lösung: Kürzere Liste mit Nullen auffüllen!

```
def __add__(self, other): # p1 + p2
    n = max(len(self.koeff), len(other.koeff))
    a = self.koeff + [0] * (n - len(self.koeff)) # auffüllen
    b = other.koeff + [0] * (n - len(other.koeff)) # auffüllen
    return Polynom([a[i] + b[i] for i in range(n)])
```

`__mul__` mit `isinstance` – verschiedene Typen

Ein Polynom kann mit einem **Skalar** (Zahl) ODER einem **anderen Polynom** multipliziert werden. Woher wissen wir was? → `isinstance()`!

```
def __mul__(self, other):
    if isinstance(other, (int, float)): # Skalar?
    if other == 0: # Sonderfall!
    return Polynom([0])
    return Polynom([a * other for a in self.koeff])
    elif isinstance(other, Polynom): # Polynom?
    # Cauchy-Produkt (siehe unten)
    ...
```

TIPP: `__rmul__` = **reverse** mul. Wird aufgerufen wenn der **linke** Operand kein `__mul__` für deinen Typ hat.

`3 * p` → `int` kennt kein `Polynom` → Python fragt das `Polynom`: `p.__rmul__(3)`

Cauchy-Produkt = Polynom-Multiplikation

ANALOGIE: Wie Langmultiplikation in der Schule! Jeder Term des ersten Polynoms wird mit **jedem** Term des zweiten multipliziert. Terme mit gleichem x-Exponent werden zusammengezählt.

Formel: Koeffizient c_k des Produkts = Summe aller $a_i \cdot b_j$ wobei $i + j = k$

```
# Beispiel: (1 + 2x) * (3 + x) = 3 + 7x + 2x^2
# c[0] = a[0]*b[0] = 1*3 = 3
# c[1] = a[0]*b[1]+a[1]*b[0] = 1*1+2*3 = 7
# c[2] = a[1]*b[1] = 2*1 = 2

c = [0] * (self.grad + other.grad + 1) # Platz machen
for i in range(self.grad + 1):
    for j in range(other.grad + 1):
        c[i + j] += self.koeff[i] * other.koeff[j]
    return Polynom(c)
```

Vergleichsoperatoren für Serie 10

Aufgabe 3 – Polynome nach betragmäßigem Wachstum vergleichen:

- Höherer Grad = größer
- Gleicher Grad → Betrag des höchsten Koeffizienten entscheidet
- Auch der gleich? → Gleichheit

```
def __le__(self, other): # p <= q
    if self.grad != other.grad:
    return self.grad <= other.grad
    return abs(self.koeff[-1]) <= abs(other.koeff[-1])

def __ge__(self, other): # p >= q
    return other.__le__(self) # einfach umdrehen!

# Aufgabe 5: TimeInterval sortieren (nur __lt__ reicht für sorted):
def __lt__(self, other): # für sorted()
    return self.start < other.start
```

TEIL 3: Fehlerkontrolle

ANALOGIE: Stell dir vor, du machst Pfannkuchen. **try** = du versuchst ihn zu wenden. **except** = er fällt auf den Boden, du reagierst. **else** = er ist perfekt gelandet, du feierst. **finally** = du räumst die Küche auf, egal was passiert ist.

Die 4 Blöcke:

Block	Wann läuft er?	Pflicht?
try:	IMMER – hier steht der riskante Code	Ja
except:	NUR wenn ein Fehler im try passiert	Ja (mind. 1)
else:	NUR wenn KEIN Fehler passiert ist	Nein (optional)
finally:	IMMER – egal ob Fehler oder nicht	Nein (optional)

```
try:
    result = 10 / x
except ZeroDivisionError: # spezifischer Fehler
    print('x darf nicht 0 sein!')
except TypeError: # anderer Fehler
    print('x muss eine Zahl sein!')
else: # alles gut gegangen
    print(f'Ergebnis: {result}')
finally: # immer
    print('Fertig.')
```

TIPP: Du kannst auch **alle** Fehler auf einmal fangen:
`except Exception as e: → print(e)` zeigt die Fehlermeldung.

Die wichtigsten Fehlertypen:

Fehlertyp	Passiert wenn...	Beispiel
ZeroDivisionError	Division durch 0	10 / 0
TypeError	Falscher Datentyp	"abc" + 5
ValueError	Richtiger Typ, falscher Wert	int("hallo")
IndexError	Index existiert nicht	[1,2][99]
KeyError	Key nicht im Dictionary	d["nope"]
AssertionError	assert ist False	assert 1 == 2

raise – Fehler SELBST werfen

Warum? Damit der Benutzer deiner Funktion sofort merkt, dass etwas falsch ist. Besser als ein `print()`, weil das Programm **stoppt**.

```
def zeros(self):
    if self.grad != 2:
        raise ValueError('Nur für Polynome 2. Grades!') # STOPP!
    # ... Nullstellen berechnen ...
```

assert – Schnellcheck

Wie ein Sicherheitscheck am Eingang. Bedingung erfüllt? → Weitergehen. Nicht erfüllt? → Raus.

```
assert isinstance(a, int) and a > 0, 'a muss positive ganze Zahl sein'
# Wenn False → AssertionError mit der Nachricht
```

	raise	assert
Fehlertyp	Du wählst (TypeError, ValueError,...)	Immer AssertionError
Wofür?	Produktionscode, Benutzer-Input	Debugging, Entwicklung
Syntax	raise TypeError("...")	assert bedingung, "..."

TEIL 4: Testen und Debugger

Zwei Arten von Tests:

Art	Frage	Beispiel
Strukturell	Läuft jeder Codepfad?	Wird der else-Zweig je erreicht?
Funktional	Stimmt das Ergebnis?	eval(2) ergibt wirklich 13?

KEY TAKEAWAY: Nach **JEDER** neuen Funktion sofort testen! Nicht erst wenn alles fertig ist. Nach jeder Korrektur **ALLE** Tests nochmal laufen lassen.

VSCode Debugger – Kurzversion:

Taste	Was passiert?
Klick links neben Zeile	Breakpoint setzen (rotes Pünktchen)
F5	Weiter zum nächsten Breakpoint
F10	Nächste Zeile (Funktion überspringen)
F11	In Funktion reinspringen
Shift+F11	Aus Funktion rausspringen

TIPP: Im Debugger kannst du in der **Debug Console** live Python-Befehle eingeben und Variablen ändern!

TEIL 5: Alles zusammen – Polynom-Klasse

Hier siehst du die komplette Klasse, die du für Aufgaben 1–3 brauchst. Lies sie **Block für Block**, nicht alles auf einmal.

Block 1: Konstruktor + Fehlerkontrolle

```
class Polynom:
    def __init__(self, koeff):
        if not isinstance(koeff, list):
            raise TypeError('Koeffizienten müssen Liste sein')
        if len(koeff) == 0:
            raise ValueError('Liste darf nicht leer sein')
        # Führende Nullen entfernen (außer Nullpolynom)
        while len(koeff) > 1 and koeff[-1] == 0:
            koeff.pop()
        self.koeff = koeff
        self.grad = len(koeff) - 1
```

Block 2: eval, deriv, zeros

```
def eval(self, x): # f(x) berechnen
    return sum(a * x**i for i, a in enumerate(self.koeff))

def deriv(self): # f'(x) als neues Polynom
    if self.grad == 0:
        return Polynom([0])
    return Polynom([i * self.koeff[i] for i in range(1, self.grad + 1)])

def zeros(self): # Nullstellen (nur Grad 2!)
    if self.grad != 2:
        raise ValueError('Nur für Grad 2')
    a, b, c = self.koeff[2], self.koeff[1], self.koeff[0]
    disk = b**2 - 4*a*c
    if disk < 0: return None
    x1 = (-b + disk**0.5) / (2*a)
    x2 = (-b - disk**0.5) / (2*a)
    return (x1, x2)
```

Block 3: __add__, __mul__, __rmul__

```

def __add__(self, other):
    n = max(len(self.koeff), len(other.koeff))
    a = self.koeff + [0]*(n - len(self.koeff))
    b = other.koeff + [0]*(n - len(other.koeff))
    return Polynom([a[i]+b[i] for i in range(n)])

def __mul__(self, other):
    if isinstance(other, (int, float)): # Skalar
    if other == 0: return Polynom([0])
    return Polynom([a*other for a in self.koeff])
    elif isinstance(other, Polynom): # Polynom
    c = [0] * (self.grad + other.grad + 1)
    for i in range(self.grad + 1):
    for j in range(other.grad + 1):
    c[i+j] += self.koeff[i] * other.koeff[j]
    return Polynom(c)

def __rmul__(self, other): #  $3 * p \rightarrow p.__rmul__(3)$ 
    return self.__mul__(other)

```

Block 4: __str__, __le__, __ge__

```

def __str__(self): # '1 + 3x^2'
    terms = []
    for i, a in enumerate(self.koeff):
    if a == 0: continue # 0-Terme weglassen!
    if i == 0: terms.append(str(a))
    elif i == 1: terms.append(f'{a}x')
    else: terms.append(f'{a}x^{i}')
    return ' + '.join(terms) if terms else '0'

def __le__(self, other): #  $p \leq q$ 
    if self.grad != other.grad:
    return self.grad <= other.grad
    return abs(self.koeff[-1]) <= abs(other.koeff[-1])

def __ge__(self, other): #  $p \geq q$ 
    return other.__le__(self)

```

Häufige Fallen in Serie 10:

ACHTUNG: Multiplikation mit 0: $\text{Polynom}([1, 2, 3]) * 0$ soll $\text{Polynom}([0])$ ergeben (Grad 0), NICHT $\text{Polynom}([0, 0, 0])$ (Grad 2)!

ACHTUNG: __str__ Nullterme: $1 + x^2$ statt $1 + 0x + x^2$. Terme mit Koeffizient 0 überspringen!

ACHTUNG: __repr__ für Listen: `print(sorted(lst))` nutzt `__repr__`, nicht `__str__`! Beide implementieren wenn nötig. Auch bei **Rückwärtseinsetzen (Aufgabe 6)**: Auf `u[i][i] == 0` prüfen → Division durch 0 abfangen mit `try/except`!